CDTA INTERNSHIP TECHNICAL REPORT

Human Activity Recognition Using Classical Machine Learning on Shimmer Sensor Data

Machine Learning Fundamentals & Transfer Learning

Host institution	Centre de Développement des Technologies Avancées (CDTA)
Domain	Machine Learning, Signal Processing, Human Activity Recognition
Implementation	Python, NumPy, Pandas, SciPy, Scikit-learn, Streamlit
Reporting period	October 2026

Documentation draft based on the project details and preliminary results discussed to date. Implementation-specific values and screenshots should be verified before formal submission.

Table of Contents

1. Abstract
2. Introduction and Problem Statement
3. Objectives and Project Scope
4. Project Environment and Structure
5. Dataset Description
6. Data Loading and Preprocessing
7. Data Quality Assessment
8. Exploratory Signal Analysis
9. Signal Windowing
10. Feature Extraction
11. Machine Learning Methodology
12. K-Nearest Neighbors
13. Support Vector Machine
14. Decision Tree
15. Hyperparameter Optimization
16. Experimental Protocol
17. Evaluation Metrics
18. Preliminary Results
19. Results Analysis
20. Streamlit Dashboard
21. Installation and Execution
22. Reproducibility
23. Limitations and Threats to Validity
24. Proposed Improvements
25. Conclusion
26. References
27. Appendices and Verification Checklist

1. Abstract

Human Activity Recognition (HAR) identifies physical activities from sensor measurements. Applications include healthcare monitoring, fall detection, rehabilitation, sports analysis, and assistive technologies. This project investigates classical machine learning methods for recognizing activities from Shimmer wearable-sensor recordings.

The discussed dataset contains seven activity classes: Fall, Jump, Walk, Run, Stand, Sit, and Lay. The pipeline loads MATLAB .mat files, inspects sensor channels, segments recordings into fixed-length windows, extracts statistical features, and evaluates K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Decision Tree classifiers.

The baseline uses windows of 128 samples and 12 features: maximum, minimum, mean, and population standard deviation for each of the three accelerometer axes. KNN hyperparameters were reportedly optimized using GridSearchCV and Matthews Correlation Coefficient (MCC) scoring.

Across ten repeated random window-level train-test splits, tuned KNN reached a mean accuracy of 96.21%, macro F1-score of 94.34%, and MCC of 0.9545. These results require validation, especially because windows from the same original recording may have appeared in both training and test partitions.

Keywords: Human Activity Recognition, Shimmer sensors, machine learning, KNN, SVM, Decision Tree, feature engineering, MCC, Streamlit.

2. Introduction and Problem Statement

2.1 Background

Wearable accelerometers measure acceleration along multiple axes; gyroscopes measure angular velocity. Their signals contain patterns related to physical movement. Walking often creates repeated oscillations, running may produce more intense and rapid changes, and stationary activities may produce more stable measurements. A classifier can learn statistical patterns from these signals and predict activity labels.

Raw sensor recordings are time series, whereas conventional machine learning algorithms generally expect fixed-length vectors. This project therefore segments signals into windows and summarizes each window using statistical features.

2.2 Problem statement

The central question is: how effectively can statistical features extracted from Shimmer recordings classify seven human activities using classical machine learning?

Supporting questions concern whether the features separate activities, which model performs best under the initial evaluation protocol, whether KNN tuning improves performance, how suitable metrics describe the results, and how a Streamlit interface can present the pipeline.

3. Objectives and Project Scope

• Inspect the MATLAB recordings, their structure, channel mapping, and activity labels.

• Assess data quality, including missing values, non-finite values, duplicates, timestamp consistency, and incomplete windows.

• Segment recordings into 128-sample windows and extract statistical features.

• Train and evaluate KNN, SVM, and Decision Tree classifiers.

• Use repeated stratified train-test splits and metrics including MCC.

• Present data and experiments through a Streamlit dashboard.

This report focuses on the classical machine learning stage. Transfer learning with architectures such as ResNet or VGG was part of the broader internship topic, but it is not claimed as completed in this sensor-classification pipeline.

4. Project Environment and Structure

The working directory discussed was C:\Users\HP\Desktop\shimmer_ml_project. The following is a suggested structure based on filenames mentioned during development; it is not a verified inventory of every file.

```
shimmer_ml_project/
├── .venv/
├── app.py
├── data_utils.py
├── feature_extraction.py
├── ml_experiments.py
├── data/
│   ├── Fall.mat
│   ├── Jump.mat
│   ├── Lay.mat
│   ├── Run.mat
│   ├── Sit.mat
│   ├── Stand.mat
│   └── Walk.mat
├── csv/
│   └── features_128.csv
├── results/
│   └── evaluation_results.csv
└── requirements.txt
```

Technology	Role
Python	Main programming language
NumPy	Numerical operations and array processing
Pandas	Tabular data and feature datasets
SciPy	Reading MATLAB .mat files
Scikit-learn	Models, preprocessing, tuning, and metrics
Matplotlib	Signal and result visualization
Streamlit	Interactive dashboard
Git	Version control, if used

5. Dataset Description

5.1 Activity classes

File	Activity label
Fall.mat	Fall
Jump.mat	Jump
Lay.mat	Lay
Run.mat	Run
Sit.mat	Sit
Stand.mat	Stand
Walk.mat	Walk

5.2 Sensor channels

The recordings were described as containing 11 columns, including timestamps, three accelerometer channels, three gyroscope channels, and additional columns. The accelerometer channels were referred to as accel_x, accel_y, and accel_z. Exact MATLAB variable names, column indices, units, and the meaning of additional columns must be verified from the source files.

Channel group	Status
Timestamp	Reported; exact column and units to verify
Accelerometer X/Y/Z	Reported; exact indices and units to verify
Gyroscope X/Y/Z	Reported; exact indices and units to verify
Additional columns	Not yet verified

5.3 Reported feature-window distribution

Activity	Windows	Percentage
Stand	397	21.39%
Walk	346	18.64%
Sit	334	18.00%
Lay	318	17.13%
Run	279	15.03%
Jump	138	7.44%
Fall	44	2.37%
Total	1,856	100%

The previously discussed feature dataset contained 1,856 rows and 15 columns: 12 numerical features, the activity label, a segment identifier, and the starting sample index. Fall is the smallest class, so overall accuracy alone may hide poor fall detection.

6. Data Loading and Preprocessing

6.1 Loading MATLAB files

```
from scipy.io import loadmat

mat_data = loadmat("Fall.mat")
print(mat_data.keys())

# Use this only if the variable is actually named "data":
data = mat_data["data"]
print("Shape:", data.shape)
print(data[:5])
```

Inspect the keys and array shape before processing. A variable named data is assumed in the example; the actual variable name must be verified for every file.

6.2 Selecting accelerometer axes

```
# Placeholder column indices: replace with verified indices.
accel_x = data[:, x_column]
accel_y = data[:, y_column]
accel_z = data[:, z_column]
```

Incorrect column indices could select timestamps, gyroscope channels, or unrelated fields. Document the mapping before running the final experiments.

6.3 Sampling frequency

For a window of N samples at sampling frequency fs, nominal window duration is T = N/fs. For N=128 and fs=50 Hz, T=2.56 seconds. At fs=51.2 Hz, T=2.50 seconds. The elapsed time between the first and last of 128 regularly spaced samples is 127/fs. Confirm the actual sampling frequency from validated timestamps rather than assuming a nominal rate.

7. Data Quality Assessment

The quality stage should examine missing values, non-finite values, duplicates, timestamp intervals, and incomplete windows. The appropriate treatment depends on the cause of each problem; do not remove data solely because sensor values repeat.

```
import numpy as np
import pandas as pd

df = pd.DataFrame(data)
print(df.isna().sum())
print("Duplicate rows:", df.duplicated().sum())
print("All values finite:", np.isfinite(data).all())
```

Timestamp differences can be calculated as $\Delta t_i = t_{i+1} - t_i$. If timestamps are in seconds and intervals are regular, the approximate sampling frequency is 1 / median(Δt). Convert milliseconds to seconds first if necessary.

Quality check	Value to report
Original sample count	Extract from source files
Missing/non-finite values	Compute per channel
Duplicate rows	Compute and interpret in context
Timestamp irregularities	Compute after verifying units
Incomplete windows	Count samples not included in full windows
Valid feature windows	1,856 in previously discussed feature table
Activity classes	7

8. Exploratory Signal Analysis

Plotting the three accelerometer axes helps verify the data and inspect differences among activities.

```
import matplotlib.pyplot as plt
import numpy as np

sample_index = np.arange(len(accel_x))
plt.figure(figsize=(12, 5))
plt.plot(sample_index, accel_x, label="Accelerometer X")
plt.plot(sample_index, accel_y, label="Accelerometer Y")
plt.plot(sample_index, accel_z, label="Accelerometer Z")
plt.title("Accelerometer Signals")
plt.xlabel("Sample")
plt.ylabel("Acceleration")
plt.legend()
plt.tight_layout()
plt.show()
```

8.1 Acceleration magnitude

```
accel_magnitude = np.sqrt(
    accel_x**2 + accel_y**2 + accel_z**2
)
```

Acceleration magnitude summarizes the combined magnitude of the three axes. In the baseline discussed here, it was intended for visualization or future experiments, not as an extra feature in the 12-feature baseline.

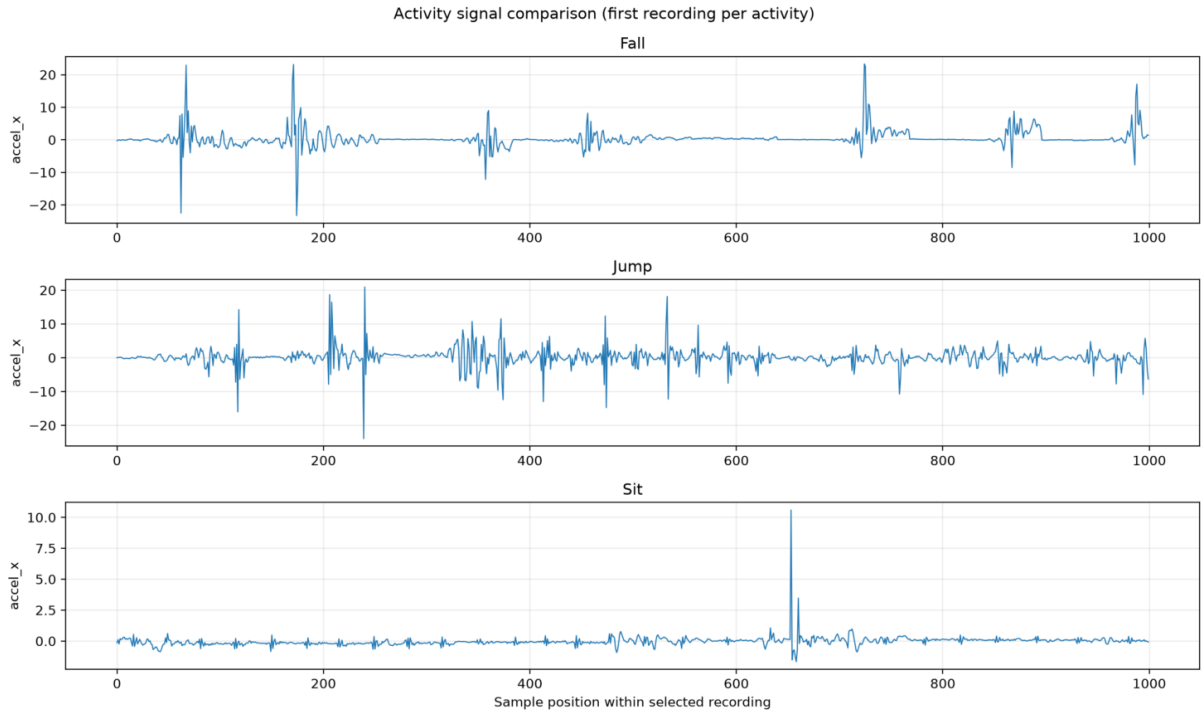


Figure 1 — accel_x signal for Fall, Jump and Sit (first recording of each activity, first 1,000 samples).

Fall shows sharp isolated spikes (up to about ±23) separated by quiet stretches, Jump shows frequent high-amplitude bursts, and Sit stays close to zero apart from one short spike. These differences in spread and extremes are what the maximum, minimum and standard-deviation features capture.

Figure placeholder: insert a real plot of acceleration magnitude for the same recording.

9. Signal Windowing

A continuous recording is converted into fixed-length segments so each segment can be represented by a feature vector. The baseline uses 128 samples per non-overlapping window.

```
window_length = 128
n_windows = len(accel_x) // window_length

for i in range(n_windows):
    start = i * window_length
    end = start + window_length
    x_window = accel_x[start:end]
    y_window = accel_y[start:end]
    z_window = accel_z[start:end]
    # Extract features and preserve the source recording label.
```

The number of complete windows is floor(N/128). A final partial window may be discarded, padded, or processed differently; document the chosen policy and the number of discarded samples.

```
Original recording
   ├── Samples 0–127       -> Window 1
   ├── Samples 128–255     -> Window 2
   ├── Samples 256–383     -> Window 3
   ├── Samples 384–511     -> Window 4
   └── Remaining samples   -> Handle by documented policy
```

10. Feature Extraction

The baseline calculates four statistics for each of three accelerometer axes: maximum, minimum, mean, and population standard deviation (ddof=0). This yields 3 × 4 = 12 features per window.

Axis	Features
X	x_max, x_min, x_mean, x_std

Axis	Features
Y	y_max, y_min, y_mean, y_std
Z	z_max, z_min, z_mean, z_std

For a window $x_1,\ldots,x_N$, the mean is $\bar{x} = (1/N)\Sigma x_i$ and population standard deviation is $\sigma = \sqrt{[(1/N)\Sigma(x_i - \bar{x})^2]}$. Maximum and minimum capture extreme values, while standard deviation measures variation.

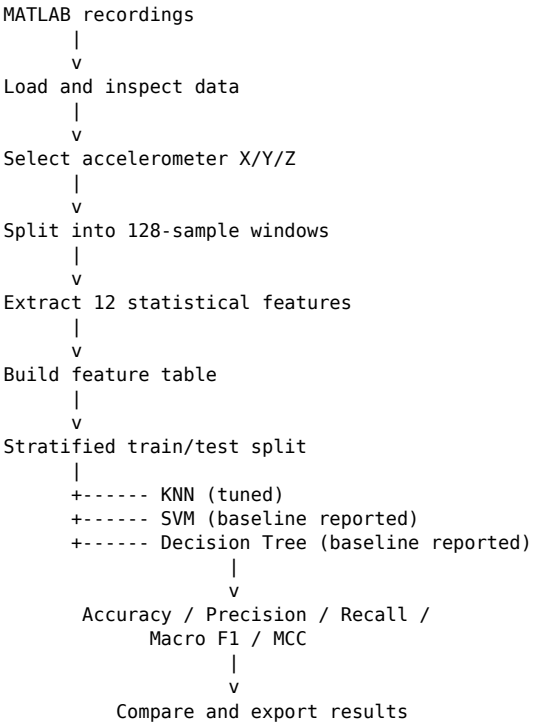
```
def extract_features(window_x, window_y, window_z):
    features = {}
    for axis, values in [
        ("x", window_x), ("y", window_y), ("z", window_z)
    ]:
        features[f"{axis}_max"] = values.max()
        features[f"{axis}_min"] = values.min()
        features[f"{axis}_mean"] = values.mean()
        features[f"{axis}_std"] = values.std(ddof=0)
    return features
```

This is an illustrative implementation of the described method, not a verified copy of the actual feature_extraction.py.

```
Feature dataset
Rows: 1,856 windows (reported)
Model inputs: 12 numerical features
Target: one of 7 activity labels
Metadata: segment ID and window start index
Total columns: 15 (reported)
```

Metadata such as segment_id and window_start_sample should not be used as model features unless explicitly justified. They are useful for traceability and for group-based splitting.

11. Machine Learning Methodology



The diagram describes the overall methodology. Exact implementation details should be checked against the source code.

12. K-Nearest Neighbors

KNN predicts the label of a new observation by finding its k nearest training observations and voting among their labels. Manhattan distance is $\Sigma|x_i - y_i|$; Euclidean distance is $\sqrt{\Sigma(x_i - y_i)^2}$.

KNN is sensitive to feature scales, so standardization is typically applied inside a scikit-learn pipeline. The scaler must be fitted only on the relevant training data.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])
```

Strengths: simple, nonlinear decision boundaries, effective when same-class points cluster together. Limitations: scale sensitivity, prediction cost, sensitivity to noisy observations, and dependence on k and distance settings.

13. Support Vector Machine

SVM learns a decision boundary intended to separate classes with a large margin. Kernel functions can represent nonlinear boundaries. The exact kernel and hyperparameters used in the project should be verified.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

svm_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC())
])
```

SVM usually benefits from feature scaling. Its performance depends on the kernel and parameters such as C and gamma. The previously reported result is a baseline result; do not describe SVM as tuned unless the code confirms it.

14. Decision Tree

A Decision Tree classifies observations through a sequence of feature thresholds. Gini impurity for class proportions p_i is $G = 1 - \Sigma p_i^2$. Trees can model nonlinear patterns and are relatively easy to visualize, but can overfit and may vary substantially with the training data.

The previously reported Decision Tree result should be treated as a baseline unless pruning or hyperparameter optimization is confirmed.

15. Hyperparameter Optimization

The discussed KNN grid searches neighbor counts from 1 to 30, uniform or distance weighting, and Minkowski p values 1 or 2. This represents $30 \times 2 \times 2 = 120$ parameter combinations. With five-fold cross-validation, it requires 600 fits per outer run before refitting.

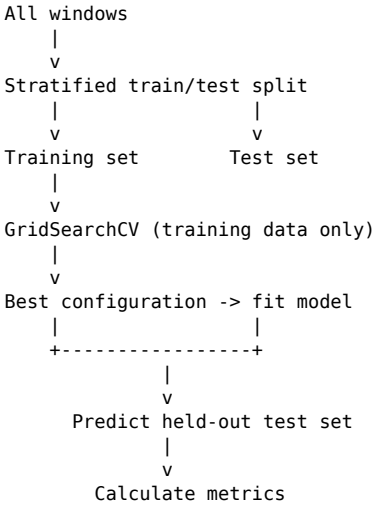
Parameter	Values
n_neighbors	1 through 30
weights	uniform, distance
p	1, 2
Cross-validation	5-fold stratified (reported)
Scoring	MCC

The test set must not be used to select parameters. The intended process is: split data; run GridSearchCV on training data only; refit the selected model on training data; evaluate once on the untouched test set.

16. Experimental Protocol

The code is run at least ten times, and up to 100 runs can be considered for research. The initial protocol used ten stratified 80/20 random window-level splits, with seeds 0–9. KNN tuning was reportedly performed only within each training partition.

For a metric m over R runs, mean $\bar{m} = (1/R)\Sigma m_r$. The sample standard deviation summarizes variability: $s = \sqrt{[\Sigma(m_r - \bar{m})^2/(R-1)]}$. Report both mean and standard deviation, not only the mean.



Important limitation: random window-level splitting can put related windows from the same recording into both partitions. A recording-level or participant-level split is preferable when identifiers are available.

17. Evaluation Metrics

17.1 Accuracy

Accuracy = correct predictions / total predictions. It is easy to interpret but may hide weak performance on minority classes.

17.2 Precision and recall

Precision = TP/(TP+FP), measuring how many predictions of a class are correct. Recall = TP/(TP+FN), measuring how many actual examples of a class are found. For fall detection, recall is important because false negatives represent missed falls.

17.3 F1-score

F1 = 2 × precision × recall / (precision + recall). Macro F1 computes the F1-score for each class and averages the scores equally, giving minority classes equal weight.

17.4 Matthews Correlation Coefficient

MCC summarizes agreement between actual and predicted labels and is useful under class imbalance. For binary classification, $MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{[(TP+FP)(TP+FN)(TN+FP)(TN+FN)]}$. Multiclass classification uses a generalized formulation. MCC ranges from −1 to +1: +1 perfect agreement, 0 no correlation-like agreement, −1 complete disagreement. MCC is not itself a formal statistical significance test.

17.5 Confusion matrix

A seven-class confusion matrix has seven rows for actual classes and seven columns for predicted classes. Diagonal entries are correct predictions; off-diagonal entries reveal misclassifications. Specific class-confusion patterns should be described only after inspecting the actual matrix.

18. Preliminary Results

Values are means over ten repeated stratified 80/20 window-level splits (± is the sample standard deviation), taken from the latest experiment summary export.

Model	Accuracy	Balanced accuracy	Macro F1	MCC
Tuned KNN	0.9621 ± 0.0089	0.9297	0.9434	0.9545 ± 0.0107
Decision Tree	0.9300 ± 0.0139	0.9165	0.9145	0.9159 ± 0.0165
SVM	0.8842 ± 0.0086	0.8938	0.8955	0.8613 ± 0.0103

Earlier run: default KNN	Value
Default KNN configuration	k = 5
Mean accuracy	0.9427
Mean MCC	0.9312

The k = 5 baseline above comes from an earlier experiment run and is not directly comparable with the table above. Re-run the baseline on the same splits before quoting a tuning gain.

18.1 Repeated-split visualizations

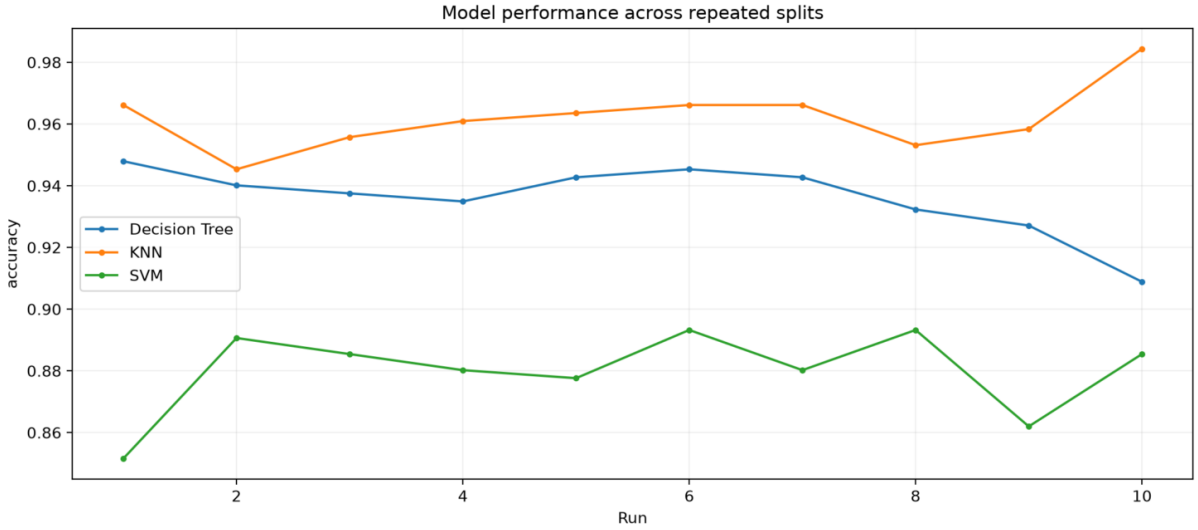


Figure 2 — Accuracy of each model across the ten repeated splits.

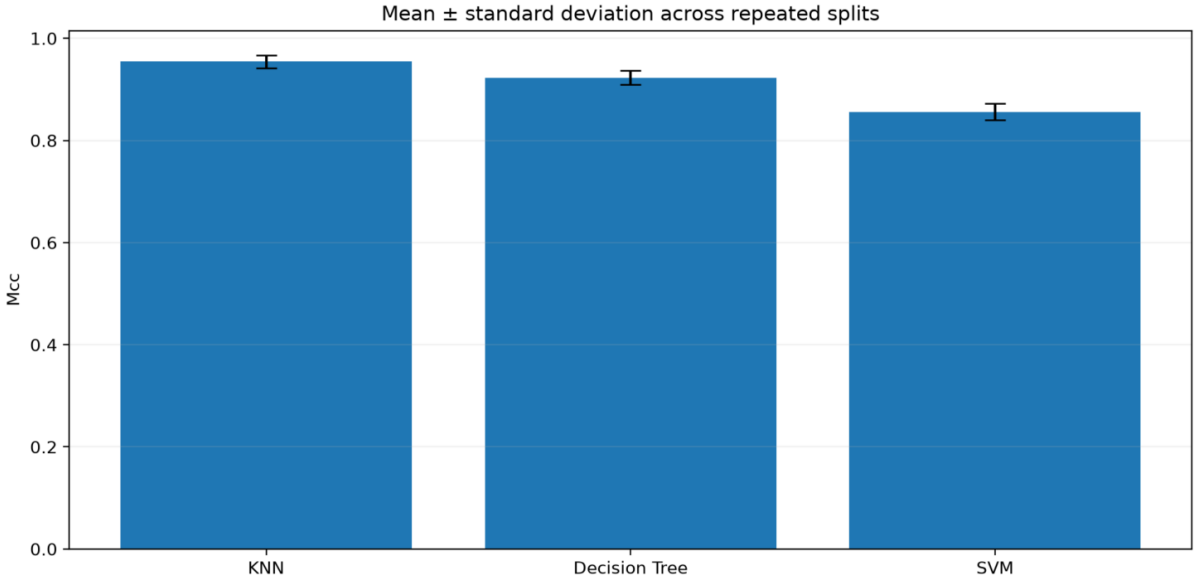


Figure 3 — Mean MCC ± standard deviation across the repeated splits.

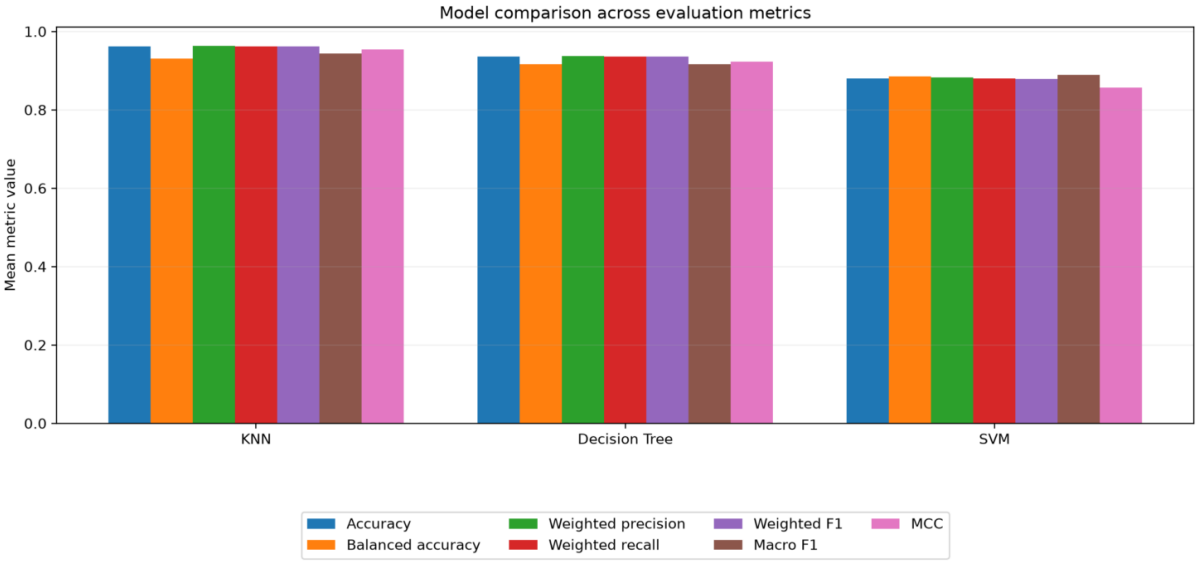


Figure 4 — Mean value of each evaluation metric per model.

18.2 Best KNN configuration

Parameter	Reported value
Number of neighbors	1
Weighting	Uniform
Minkowski p	1 (Manhattan distance)

This configuration was selected in all ten runs of the earlier experiment; confirm it against the latest knn_best_parameters.csv. k=1 can be sensitive to noise, so it should be validated with a group-based split.

18.3 One reported split: Fall and Jump

Class	Correct / actual	Approx. recall
Fall	7 / 9	77.8%
Jump	24 / 28	85.7%

These counts refer to one reported split, not the ten-run mean. Recall based on nine Fall examples is highly uncertain.

Figure placeholder: insert a confusion matrix generated from the actual held-out predictions.

19. Results Analysis

The reported results suggest that the 12 statistical features contain useful information for distinguishing the seven activity classes under the initial protocol. KNN had the highest reported mean accuracy and MCC. Potential reasons include locally separable feature clusters and the effect of hyperparameter tuning, but these are hypotheses rather than proven causes.

In the run-by-run plot, KNN has the highest accuracy in all ten splits and SVM the lowest. For KNN and the Decision Tree, balanced accuracy and macro F1 sit below overall accuracy, which points to weaker recognition of some minority classes; SVM shows the opposite pattern.

The SVM result may reflect untuned parameters, a kernel that does not match the feature geometry, or other pipeline differences. The Decision Tree provides a nonlinear, interpretable baseline but may overfit. A fair comparison should tune all three models within training-only cross-validation and evaluate them on identical outer splits.

For healthcare-related use, inspect per-class precision, recall, F1, the number of missed falls, false alarms, and performance on independent recordings. Aggregate metrics alone are not enough.

20. Streamlit Dashboard

The discussed dashboard is intended to present dataset information, raw signals, experiments, model comparisons, and downloads. The exact current implementation must be verified against app.py.

Section	Purpose
Overview	Dataset metrics, activity distribution bar chart, workflow
Data Quality	Text/tables for missing values, duplicates, timestamps, warnings
Raw Signals	Inspect accelerometer channels by activity or recording
Model Experiments	KNN, SVM, Decision Tree results and configurations
Model Comparison	Compare metrics using a consistent evaluation protocol
Downloads	Export features and experiment results

The intended UI update was to remove the EDA tab and circular/pie chart while retaining the Overview metrics, activity distribution bar chart, tables, workflow, Raw Signals, model experiments, comparison, and downloads. Data Quality was intended to be text and tables only. Confirm these edits in the running application.

Suggested screenshots

• Screenshot 1 — Overview showing dataset metrics and activity distribution bar chart.

• Screenshot 2 — Data Quality section showing quality tables and warnings.

• Screenshot 3 — Raw Signals section with three accelerometer channels.

• Screenshot 4 — Model comparison section.

• Screenshot 5 — KNN experiment details and best parameters.

• Screenshot 6 — Confusion matrix from actual test predictions.

These are screenshot placeholders, not images of the live application. Insert genuine screenshots before formal submission.

21. Installation and Execution

21.1 Open the project folder

```
cd C:\Users\HP\Desktop\shimmer_ml_project
```

21.2 Activate the virtual environment

```
.\.venv\Scripts\Activate.ps1
```

If PowerShell blocks activation, use the environment's Python executable directly.

21.3 Install dependencies

```
python -m pip install -r requirements.txt
```

If there is no requirements.txt, the packages relevant to this report can be installed with:

```
python -m pip install numpy pandas scipy scikit-learn matplotlib streamlit
```

21.4 Run the dashboard

```
python -m streamlit run app.py
```

The application usually opens at http://localhost:8501. If the standalone streamlit command is not recognized, python -m streamlit is a useful alternative.

21.5 Verify the environment

```
python --version
python -m pip --version
python -m pip show streamlit
python -m pip show scikit-learn
python -m pip freeze > environment.txt
```

22. Reproducibility

Preserve original filenames, sample counts, channel mapping, sampling frequency, labels, quality checks, window size and overlap, incomplete-window policy, feature definitions, model parameters, random seeds, cross-validation configuration, and package versions.

The preferred experiment output contains one row per model per run, with accuracy, precision, recall, macro F1, MCC, and model configuration. Report mean and standard deviation across runs.

Run	Model	Accuracy	Precision	Recall	Macro F1	MCC
0	KNN	Insert	Insert	Insert	Insert	Insert
0	SVM	Insert	Insert	Insert	Insert	Insert
0	Decision Tree	Insert	Insert	Insert	Insert	Insert
1	KNN	Insert	Insert	Insert	Insert	Insert

23. Limitations and Threats to Validity

23.1 Potential window-level leakage

If related windows from the same original recording are randomly divided between training and testing, shared recording conditions may inflate performance. A group-based split keeps all windows from one recording together. If participant identifiers exist, participant-level evaluation can provide stronger evidence of generalization to new people.

```
Illustration of a risky split:
Recording A -> Window 1: train; Window 2: train;
               Window 3: test;  Window 4: test

Preferred principle:
Recording A -> training only
```

```
Recording B -> training only
Recording C -> test only
```

23.2 Small Fall class

Only 44 Fall windows were present in the reported feature table. More independent fall recordings are needed for reliable fall-specific estimates.

23.3 Limited feature representation

The 12-feature baseline does not fully capture temporal structure, frequency-domain information, or relationships between channels. Possible additions include RMS, energy, interquartile range, skewness, kurtosis, cross-axis correlation, spectral energy, and spectral entropy. Evaluate these as separate experiments.

23.4 Unequal tuning effort

Tuned KNN was compared with SVM and Decision Tree baseline results. This does not establish that KNN is inherently superior. Tune all models using training-only cross-validation for a fair comparison.

23.5 Uncertainty

The reported means alone do not permit calculation of standard deviations, confidence intervals, or a defensible significance test. Individual run results are required.

23.6 Real-world generalization

Sensor placement, user differences, activity execution, environment, and sensor noise can affect performance. High random-split scores do not establish clinical validity.

24. Proposed Improvements

Strengthen evaluation

Use recording-level or participant-level group splits, keep tuning inside training data, and evaluate on an independent test set.

Tune all models

Tune KNN, SVM, and Decision Tree under the same cross-validation protocol.

Add richer features

Evaluate magnitude, RMS, energy, correlations, and frequency-domain descriptors in controlled experiments.

Analyze class-specific performance

Report per-class precision, recall, F1, confusion matrices, missed falls, and false alarms.

Investigate transfer learning

Choose a model compatible with the sensor modality; image architectures require an appropriate representation and validation.

Improve the dashboard

Show provenance, model configurations, per-class metrics, exportable results, and a clear warning about window-level splitting.

These are future-work recommendations, not claims that the tasks have already been implemented.

25. Conclusion

This project establishes a classical machine learning baseline for Human Activity Recognition from Shimmer sensor data. The described workflow includes loading MATLAB recordings, selecting accelerometer axes, windowing, extracting 12 statistical features, training KNN/SVM/Decision Tree classifiers, evaluating metrics, and presenting results through a Streamlit dashboard.

Across ten repeated random window-level splits, tuned KNN reached mean accuracy 96.21%, macro F1 94.34%, and MCC 0.9545, ahead of the Decision Tree (accuracy 93.00%) and SVM (88.42%). These results are promising but should not be treated as definitive evidence of real-world performance until validated using group-based splits and complete run-level statistics.

The most important next step is to prevent windows from the same recording appearing in both training and testing, then repeat the model comparison with a consistent protocol. Additional work should focus on Fall recall, independent recordings, and reproducible reporting.

26. References

Scikit-learn — KNeighborsClassifier: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

Scikit-learn — SVC: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

Scikit-learn — Decision Trees: https://scikit-learn.org/stable/modules/tree.html

Scikit-learn — GridSearchCV: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

Scikit-learn — Matthews Correlation Coefficient: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.matthews_corrcoef.html

Scikit-learn — Cross-validation: https://scikit-learn.org/stable/modules/cross_validation.html

SciPy — loadmat: https://docs.scipy.org/doc/scipy/reference/generated/scipy.io.loadmat.html

Streamlit documentation: https://docs.streamlit.io/

These references explain the software and methods; they do not establish the provenance or clinical validity of the specific dataset.

27. Appendices and Verification Checklist

27.1 Final verification checklist

☐ Confirm the exact MATLAB variable name in every file.

☐ Confirm all 11 column mappings and measurement units.

☐ Confirm the actual sampling frequency.

☐ Confirm window overlap and incomplete-window handling.

☐ Verify that the 12 feature columns match the report.

☐ Confirm the CSV filename used by extractor and trainer.

☐ Retrieve individual results from all ten experiments.

☐ Calculate standard deviations for each metric.

☐ Confirm whether SVM and Decision Tree were tuned.

☐ Save confusion matrices and per-class metrics.

☐ Investigate recording-level or participant-level leakage.

☐ Insert screenshots from the real application and signal plots.

☐ Record Python and package versions.

☐ Label preliminary results and future work clearly.

Final note: Before presenting the 96.2% KNN accuracy as a research conclusion, validate it with a split that keeps related windows from the same recording together. This is more important than simply increasing the number of random window-level runs.